

MCDI500

PROGRAMACIÓN PARA LA

CIENCIA DE DATOS

AVANCE DEL PROYECTO

FASE 3 – SEMANA 2

Nombre integrante/s:

Verónica Durán Cisterna

Raúl Moya Arriagada

Daniela Rojas Vilches

Manuel Sánchez Cárcamo

Curso: MCDI500 - Programación para la Ciencia de Datos

Docente: Dr. Omar Salinas Silva

URL repositorio:

<https://github.com/vedurancisterna-alt/Proyecto-generacion-electrica-centrales-CMPC>

Fecha: 27 de septiembre de 2026

I. Índices

Índice General

I. Índices.....	2
Índice General	2
Índice de Tablas	3
II. Codificación funcional y arquitectura básica del script.....	4
2.1 Continuidad $F1 \rightarrow F2 \rightarrow F3$	4
2.2 Organización modular	4
2.3 Flujo general de datos.....	5
III. Preprocesamiento y transformación del dataset	6
3.1 Fuente y delimitación	6
3.2 Transformación ancho $\rightarrow$ largo.....	7
3.3 Variables derivadas y registros iguales a cero	7
IV. Validación técnica y verificación del código	8
4.1 Controles del dataset procesado	8
4.2 Casos normales, límite y excepciones.....	8
V. Eficiencia y optimización	10
5.1 Transformación vectorizada frente a bucle.....	10
5.2 Iteración frente a recursividad en la detección de secuencias consecutivas de generación igual a cero.....	10
5.3 Benchmark sobre el volumen completo del dataset.....	11
VI. Diseño estructurado del código.....	11
6.1 Reorganización modular, separación de tareas y reutilización	11
6.2 Criterio de diseño para el uso (o no uso) de recursividad y análisis de secuencias.....	12
VII. Programación orientada a objetos	13
7.1 Herencia, polimorfismo, encapsulamiento y abstracción	13
7.2 Patrón Strategy para agregación temporal.....	16
VIII. Documentación de arquitectura	16
8.1 Arquitectura lógica.....	16
IX. Repositorio GitHub	16
X. Notebooks ejecutables	17
XI. Bibliografía.....	18
Anexo A. Síntesis de resultados técnicos	19

Índice de Tablas

Tabla 1. Componentes principales de la arquitectura implementada en F3.....	5
Tabla 2. Flujo general de los datos del proyecto.....	6
Tabla 3. Resumen de validaciones del dataset procesado.....	7
Tabla 4. Cantidad de valores 0 del dataset y su peso por central.	8
Tabla 5. Resumen de las condiciones estructurales y calidad de los datos procesados.....	8
Tabla 6. Pruebas controladas de evaluación de los algoritmos de detección de secuencias con generación igual 0 MWh.	9
Tabla 7. Caracterización de las secuencias consecutivas con generación igual a 0 MWh en las tres centrales de análisis.	9
Tabla 8. Resultados de la comparación entre Transformación Ancho->Largo vectorizada (pandas.melt) versus iterativa (iterrows).....	10
Tabla 9. Comparación de implementaciones iterativa y recursiva para la detección de secuencias consecutivas de generación igual a 0 MWh.....	11
Tabla 10. Comparación de implementaciones iterativa y vectorizada para la detección de secuencias consecutivas de generación igual a 0 MWh.....	11
Tabla 11. Estadística descriptiva de las secuencias consecutivas con generación igual a 0 MWh.....	13
Tabla 12. Evidencia de principios de POO en el proyecto.....	¡Error! Marcador no definido.
Tabla 13. Estrategias temporales implementadas y dimensiones resultantes.	16
Tabla 14. Aportes al repositorio en Fase 3.....	17
Tabla 15. Notebooks ejecutables en repositorio.	17
Tabla 16. Síntesis de resultados técnicos Fase 3.....	19

II. Codificación funcional y arquitectura básica del script

2.1 Continuidad F1 → F2 → F3

El desarrollo de la Fase 3 se fundamenta en la reutilización y consolidación del trabajo validado en Fases 1 y 2. En la Fase 1 se definió la problemática, la pregunta de investigación y el entorno técnico; en la Fase 2 se implementó y validó el pipeline de carga, preparación, transformación y validación. En esta Fase 3, se reorganiza este flujo en un núcleo algorítmico estructurado bajo el paradigma de Programación Orientada a Objetos (POO), incorporando algoritmos de búsqueda secuencial, medición de complejidad temporal y espacial, y estrategias intercambiables de agregación temporal. Esta continuidad responde a la exigencia de integrar los avances previos en un núcleo algorítmico trazable, reproducible y eficiente (Universidad Andrés Bello, 2026b).

La pregunta de investigación se mantiene: *¿Qué patrones temporales de generación eléctrica caracterizan a las centrales TER CMPC Laja, TER CMPC Pacífico y TER CMPC Santa Fe durante el período enero–agosto de 2026?* La Fase 3 no pretende establecer causas operacionales, sino construir y evaluar procedimientos que permitan caracterizar los registros temporales de forma consistente.

2.2 Organización modular

La arquitectura separa responsabilidades en módulos específicos dentro de la carpeta src/. Esto permite “limpiar” el notebook F3, dejándole un rol de director que orquesta el análisis, documenta las decisiones y muestra las evidencias, es decir, actúa como capa de integración y evidencia, mientras los módulos encapsulan operaciones específicas, descritas en la Tabla N°1. Esta separación evita concentrar toda la lógica en un único notebook, evita repetir código, facilita pruebas, reutilización, mantenimiento y auditoría.

Módulo	Componentes Principales	Entradas (Parámetros)	Salidas (Retorno)	Control defensivo/excepciones
carga.py	cargar_datos_cen()	ruta_archivo (str o Path)	DataFrame con datos crudos del CEN	FileNotFoundError si el archivo no existe en el sistema de archivos.
transformacion.py	filtrar_periodo() filtrar_centrales() transformar_ancho_largo() crear_dia_semana() crear_id_observacion()	datos (DataFrame), listas de entidades (centrales, columnas_base, columnas_hora) o rangos de fecha (fecha_inicio, fecha_fin)	Copia independiente (df.copy()) transformada a formato ancho/largo o con columnas derivadas añadidas	ValueError ante fechas invertidas o subconjuntos vacíos; KeyError ante columnas ausentes; conversión estricta con errors="raise"
validacion.py	validar_sin_nulos()	datos (DataFrame) y	Conteo de inconsistencias	Aserciones automáticas (assert) que detienen el

	validar_sin_duplicados() validar_no_negativos() validar_granularidad() validar_categorias_esperadas() validar_dataset_procesado()	umbrales parametrizados (columnas, claves, esperado=24, etc.)	detectadas (int), Series de frecuencias o diccionario resumen	flujo ante nulos, duplicados, inconsistencias horarias o negativos.
nucleo_poo.py	Clase base: Transformador; Subclases: FiltradorPeriodo, FiltradorCentrales, TransformadorAnchoLargo, ConstructorVariablesDerivadas; Orquestador: Pipeline	Parámetros de inicialización de cada paso y df (DataFrame) en ajustar() y transformar()	Objeto transformador (self) en ajuste y nuevo DataFrame transformado sin mutación de entrada	RuntimeError si se intenta transformar sin previo ajuste (_ajustado=False); KeyError si faltan variables requeridas
agregacion_temporal.py	EstrategiaAgregacion AgregacionHoraria AgregacionDiaria AgregacionMensual AgregacionPorDiaSemana CaracterizadorTemporal	Dataset procesado en formato largo (df) y objeto estrategia inyectado	Series agregada temporalmente o DataFrame comparativo unstackeado	NotImplementedError si una clase concreta omite implementar agregar().
secuencias.py	secuencias_cero_iterativa() ; secuencias_cero_rekursiva() comparar_implementaciones(); analizar_secuencias_por_central()	Secuencia numérica de generación (valores) o DataFrame con fecha, hora y central	Lista de longitudes de ceros consecutivos (list[int]), diccionario resumen o DataFrame de benchmark	TypeError ante datos no numéricos o None; ValueError si el tamaño supera el límite seguro de recursión en pila (maximo_seguro).
exportacion.py	exportar_dataset()	datos (DataFrame), ruta_salida (str o Path)	Objeto Path del archivo exportado	Creación defensiva de directorios padre (mkdir(parents=True, exist_ok=True)).

Tabla 1. Componentes principales de la arquitectura implementada en F3.

2.3 Flujo general de datos

La Tabla N°2 resume el flujo general de los datos. La estructura del pipeline como una secuencia, donde la salida validada de una etapa corresponde con la entrada de la etapa siguiente, garantiza reproducibilidad y auditabilidad en cada paso. Esta secuencia asegura que cualquier anomalía en los datos sea detectada y contenida en su etapa correspondiente, dejando un dataset analítico limpio para la fase de algoritmos y métricas, y se encuentra coordinada en la clase Pipeline.

Etapas	Entrada	Ejecutor	Operación	Salida	Volumen
1	data/raw/*.csv	cargar_datos_cen()	Lectura delimitada por ;, decimal , y UTF-8	DataFrame crudo inmutable.	359.891 filas × 33 columnas.
2	DataFrame crudo	FiltradorPeriodo	Filtrado entre 2026-01-01 y 2026-08-31	Subconjunto temporal	Período de 8 meses
3	Subconjunto temporal	FiltradorCentrales	Filtrado categórico a centrales Laja, Pacífico y Santa Fe	Registros en formato ancho.	729 filas × 33 columnas
4	729 registros anchos	TransformadorAnchoLargo	Normalización tabular mediante pandas.melt sobre 24 columnas horarias.	Registros en formato largo.	17.496 filas × 11 columnas
5	Registros formato largo	ConstructorVariablesDerivadas	Estandarización de nombres (Grupo_Reporte) y derivación de Dia_Semana e ID_Observacion	Dataset analítico consolidado.	17.496 filas × 13 columnas
6	Dataset consolidado	validar_dataset_procesado()	Auditoría de integridad: 0 nulos, 0 duplicados, 0 negativos y 24 obs/fecha	Dataset formalmente verificado.	17.496 observaciones validadas.
7	Dataset validado	CaracterizadorTemporal y secuencias.py	Inyección polimórfica de agregaciones horaria/mensual y detección de rachas de ceros.	Perfiles de generación y tablas comparativas	Tablas agregadas (72, 729, 24 y 21 filas).

Tabla 2. Flujo general de los datos del proyecto.

III. Preprocesamiento y transformación del dataset

3.1 Fuente y delimitación

Se recuerda que la fuente corresponde a Generación Real publicada por el Coordinador Eléctrico Nacional (CEN). El archivo bruto incorporado al repositorio contiene 359.891 registros de datos y 33 columnas. En Fase 3 el alcance se explicita mediante FiltradorPeriodo con fecha inicial 2026-01-01 y fecha final 2026-08-31, seguido por FiltradorCentrales para TER CMPC LAJA, TER CMPC PACIFICO y TER CMPC SANTA FE (Coordinador Eléctrico Nacional, 2022, 2026).

3.2 Transformación ancho → largo

La fuente original del CEN registra 24 mediciones horarias en columnas separadas para cada día. Para poder analizar estos datos como verdaderas series temporales, se decide utilizar `pandas.melt` en `transformar_ancho_largo()`. Esta transformación "aplana" la tabla y consolida la unidad de análisis en el formato central-fecha-hora, pasando de 729 registros a 17.496 observaciones (3 centrales × 243 días × 24 horas) y 13 columnas. El impacto de cada paso se indica en la Tabla N°2, en las etapas 2, 3 4 y 5. Esta transformación conserva la trazabilidad temporal y permite aplicar operaciones con una estructura tabular normalizada (The pandas development team, 2026).

La Tabla N°3 resume las validaciones aplicadas al dataset procesado.

Característica	Resultado validado
Centrales	3
Fechas por central	243
Combinaciones central-fecha	729
Observaciones por combinación	24
Observaciones finales	17.496
Variables finales	13
Unidad de análisis	Central + fecha + hora
Variable objetivo	Generacion_MWh

Tabla 3. Resumen de validaciones del dataset procesado.

3.3 Variables derivadas y registros iguales a cero

Se derivan las variables `Dia_Semana` e `ID_Observacion` mediante `ConstructorVariablesDerivadas`, que reutiliza `crear_dia_semana()` y `crear_id_observacion()` desde `transformacion.py`, reduciendo duplicación de código entre la lógica funcional de F2 y la capa POO de F3.

Otro punto crítico en este proyecto es el tratamiento de los 4.438 registros con generación igual a 0 MWh (un 25,37% del total). Se decide conservarlos intactos como observaciones válidas, pues desde un punto de vista técnico, la fuente de datos no ofrece información suficiente para asumir que son errores, detenciones o fallas. Eliminarlos o imputarlos con un promedio habría alterado la realidad operacional de las plantas. Además, se toma la decisión técnica de **no aplicar escalamiento ni normalización** a la variable `Generacion_MWh`, ya que los algoritmos que se desarrollan en esta fase no dependen de distancias geométricas, por lo que mantener la escala real en MWh preserva la interpretación física del problema.

Central	Ceros	Observaciones	% de ceros
TER CMPC LAJA	3.193	5.832	54,75%
TER CMPC PACIFICO	371	5.832	6,36%
TER CMPC SANTA FE	874	5.832	14,99%
Total	4.438	17.496	25,37%

Tabla 4. Cantidad de valores 0 del dataset y su peso por central.

IV. Validación técnica y verificación del código

4.1 Controles del dataset procesado

La validación técnica comprueba condiciones estructurales y de calidad antes de ejecutar los algoritmos. El dataset final no presenta valores faltantes, duplicados exactos, identificadores duplicados ni generación negativa. Cada combinación central-fecha posee exactamente 24 observaciones horarias. Esto garantiza la integridad del resultado del pipeline.

Control	Resultado
Valores faltantes	0
Duplicados exactos	0
ID_Observacion duplicados	0
Generación negativa	0
Granularidad	729 combinaciones; todas con 24 horas
Cobertura temporal	01-01-2026 a 31-08-2026
Centrales	Laja, Pacífico y Santa Fe

Tabla 5. Resumen de las condiciones estructurales y calidad de los datos procesados.

4.2 Casos normales, límite y excepciones

Dado que el dataset contiene una cantidad considerable de registros con generación igual a 0 MWh, se validan los algoritmos de caracterización de secuencias (`secuencias_cero_iterativa()` y `secuencias_cero_rekursiva()`), que se encuentran en `secuencias.py`, ante un caso normal y tres escenarios

escenarios límites mediante pruebas controladas. La Tabla N°6 describe describe estas pruebas, su entrada y los resultados esperados.

Tipo de prueba	Entrada o condición	Resultado esperado
Caso normal	Serie con ceros intercalados	Detectar correctamente cada secuencia, su inicio, fin y longitud.
Caso límite 1	Serie sin ceros	Retornar ausencia de secuencias.
Caso límite 2	Serie compuesta solo por ceros	Retornar una única secuencia con la longitud total.
Caso límite 3	Serie vacía	Resolver el caso sin generar resultados espurios.
Excepción controlada	Entrada de tipo incompatible	Rechazar explícitamente la entrada no válida.

Tabla 6. Pruebas controladas de evaluación de los algoritmos de detección de secuencias con generación igual 0 MWh.

Una vez comprobada la equivalencia funcional, se aplica el algoritmo iterativo al dataset procesado, ya que no depende de la profundidad de recursión de Python y puede procesar la totalidad de las observaciones de cada central.

Central	Numero_ secuencias	Duración_maxima _horas	Duración_promedio _horas	Total_horas_cero _en_secuencias
TER CMPC LAJA	211	375	15,132701	3.193
TER CMPC PACIFICO	76	57	4,881579	371
TER CMPC SANTA FE	196	356	4,459184	874
Totales	483	788	24.473464	4.438

Tabla 7. Caracterización de las secuencias consecutivas con generación igual a 0 MWh en las tres centrales de análisis.

En conjunto, las tres centrales acumularon 4.438 observaciones con generación igual a 0 MWh, valor consistente con la validación realizada durante la preparación de los datos. Los resultados muestran que los patrones temporales de generación igual a cero difieren entre las centrales tanto en frecuencia como en duración.

Estos resultados son descriptivos. A partir de los datos disponibles no es posible determinar las causas operacionales que originaron las secuencias de generación igual a 0 MWh.

V. Eficiencia y optimización

5.1 Transformación vectorizada frente a bucle

Se comparó la transformación ancho->largo vectorizada (`pandas.melt`), frente a una alternativa basada en `iterrows` (iterativa). Ambas producen el mismo resultado, lo cual verifica su equivalencia funcional.

Se utilizó `timeit` para medir tiempo y `tracemalloc` para medir memoria consumida en la transformación. La Tabla N°8 resume los resultados de la comparación, evidenciando que la transformación vectorizada fue más óptima y eficiente.

Implementación	Tiempo (s)	Memoria (MB)	Equivalencia
<code>pandas.melt</code>	0,069014	4,3972	Sí
<code>iterrows</code>	0,363577	11,1309	Sí

Tabla 8. Resultados de la comparación entre Transformación Ancho->Largo vectorizada (`pandas.melt`) versus iterativa (`iterrows`).

5.2 Iteración frente a recursividad en la detección de secuencias consecutivas de generación igual a cero.

La identificación de secuencias consecutivas de registros con generación igual a 0MWh es un problema directamente relacionado con la pregunta de investigación, y por tanto, es pertinente la comparación entre los algoritmos recursivo e iterativo que las caracterizan.

Se compararon los algoritmos `secuencia_cero_iterativa()` y `secuencia_cero_recursiva()` desde los puntos de vista funcional, temporal y espacial, y con ello determinar cuál es más adecuado utilizar, de acuerdo al volumen de datos a analizar. Debido al límite de profundidad de recursión de Python, la comparación de las implementaciones se evalúa sobre tamaños controlados, con valores de n iguales a 100, 250, 500 y 750. Cabe recordar que la implementación iterativa no presenta esta restricción y puede utilizarse sobre la serie completa.

Ambas alternativas presentan equivalencia funcional y complejidad temporal $O(n)$, pero la versión recursiva utiliza memoria adicional para n mayores debido a las llamadas sucesivas a la pila de ejecución, lo cual es coherente con el análisis de complejidad (Cormen et al., 2022).

N	Iterativo (s)	Recursivo (s)	Mem. iter. (B)	Mem. rec. (B)	Equiv.
100	0,000187	0,000120	1.248	1.248	True
250	0,000291	0,000419	2.448	2.448	True
500	0,000488	0,000675	4.448	8.824	True

N	Iterativo (s)	Recursivo (s)	Mem. iter. (B)	Mem. rec. (B)	Equiv.
750	0,000823	0,001083	6.448	18.952	True

Tabla 9. Comparación de implementaciones iterativa y recursiva para la detección de secuencias consecutivas de generación igual a 0 MWh.

5.3 Benchmark sobre el volumen completo del dataset

Sobre las 17.496 observaciones totales del dataset procesado se comparó, adicionalmente, una implementación iterativa con una operación vectorizada, basada en pandas, para identificar secuencias de registros iguales a 0 MWh. Esta comparación permite analizar la eficiencia sobre el volumen real del proyecto sin la restricción de profundidad de pila presente en la implementación recursiva.

Los resultados reflejados en la Tabla N°10 evidencian un compromiso entre tiempo y espacio. Para el volumen completo del dataset procesado la implementación vectorizada presenta una velocidad comparativa considerablemente menor, mientras que la implementación iterativa registra un consumo de memoria ligeramente menor. La vectorización se presenta como una alternativa atractiva y ventajosa considerando que la diferencia de memoria observada es reducida frente a la mejora temporal obtenida por ésta.

Se debe tener en consideración que los valores absolutos de tiempo y memoria dependen del entorno de ejecución, por lo cual, estas mediciones se interpretan como una comparación reproducible entre ambas implementaciones bajo las mismas condiciones y no como valores universales de rendimiento. Tracemalloc mide asignaciones gestionadas por Python, por lo que estas cifras pueden no representar necesariamente toda la memoria nativa del proceso (Python Software Foundation, 2026b).

Implementación	Registros	Tiempo promedio (s)	Memoria máxima (B)
Iterativa	17.496	0,003597	153.928
Vectorizada	17.496	0,000344	158.456

Tabla 10. Comparación de implementaciones iterativa y vectorizada para la detección de secuencias consecutivas de generación igual a 0 MWh.

VI. Diseño estructurado del código.

6.1 Reorganización modular, separación de tareas y reutilización

La estructura del proyecto organiza el trabajo separando las operaciones en archivos independientes dentro de la carpeta src/ (Tabla N°1), dejando el notebook como una guía que ejecuta y muestra los resultados. Esta decisión busca cumplir con dos principios básicos de diseño de software, vistos en el material docente de la Fase 3:

1. **Alta cohesión:** Cada archivo tiene una única tarea o responsabilidad específica. Por ejemplo, `carga.py` solo se encarga de abrir el archivo de datos; `transformacion.py` contiene funciones para transformar el formato de las tablas del dataset.
2. **Bajo acoplamiento:** Los módulos se comunican entre sí mediante entradas y salidas bien definidas, sin mezclarse innecesariamente, reduciendo la dependencia mutua. Esto implica que si en un futuro fuese necesario cambiar una regla o agregar, por ejemplo, una nueva agrupación temporal, sólo se debe modificar un archivo específico (como `agregacion_temporal.py`), sin necesidad de modificar los otros componentes ni reescribir el cuaderno principal. Así mismo, esta independencia asegura la **escalabilidad del sistema**: extender el análisis a períodos futuros o incorporar nuevas plantas generadoras sólo exige ajustar los parámetros de entrada en `FiltradorPeriodo` y `FiltradorCentrales`, sin modificar la estructura interna del pipeline.

Además, este diseño evita repetir código. Evidencias de reutilización y separación de responsabilidades podemos ver, por ejemplo, en el módulo `núcleo_poo.py`, donde en vez de reescribir desde cero la lógica de transformación de formato ancho->largo o la lectura de archivos, la nueva arquitectura importa y delega esas tareas a las funciones que ya fueron definidas y validadas en `transformacion.py` y `carga.py`.

6.2 Criterio de diseño para el uso (o no uso) de recursividad y análisis de secuencias

De acuerdo al material docente de esta Fase, la recursividad es pertinente cuando un problema presenta niveles jerárquicos variables o una profundidad desconocida al momento de escribir el código. En cambio, cuando la secuencia de pasos es fija y conocida, una solución iterativa o modular ofrece mayor claridad y menor sobrecarga.

A partir de este criterio, el proyecto adoptó dos determinaciones arquitectónicas:

1. **Diseño funcional para el pipeline:** El preprocesamiento de datos consta de cuatro etapas predeterminadas y lineales (`FiltradorPeriodo`, `FiltradorCentrales`, `TransformadorAnchoLargo` y `ConstructorVariablesDerivadas`). Introducir recursividad en este flujo habría constituido una complejidad innecesaria; por ello se estructuró mediante clases separadas.
2. **Desarrollo algorítmico en secuencias.py:** La identificación de observaciones consecutivas con generación igual a 0 MWh se utilizó como caso de estudio algorítmico. Para resolver este caso se implementaron dos variantes equivalentes: una iterativa y una recursiva. Desde la perspectiva de diseño, la función recursiva requiere reservar marcos en la pila de llamadas del sistema, lo que restringe su aplicación sobre secuencias extensas debido al límite de profundidad de Python.

Por esta razón, la alternativa recursiva se conservó como demostración conceptual sobre muestras controladas, mientras que la solución iterativa fue la seleccionada para procesar la serie temporal completa de las tres centrales (la comparación detallada de tiempos y consumo de memoria entre ambas se presenta en la Sección V). El resultado descriptivo obtenido sobre las observaciones de 0 MWh se resume en la Tabla N°11.

Central	Secuencias	Máx. (h)	Promedio (h)	Horas cero
TER CMPC LAJA	211	375	15,13	3.193
TER CMPC PACIFICO	76	57	4,88	371
TER CMPC SANTA FE	196	356	4,46	874
Totales	483	788	24,47	4.438

Tabla 11. Estadística descriptiva de las secuencias consecutivas con generación igual a 0 MWh.

VII. Programación orientada a objetos

7.1 Herencia, polimorfismo, encapsulamiento y abstracción

La Programación Orientada a Objetos se incorporó de manera aplicada al proyecto. El módulo `src/nucleo_poo.py` define una clase base `Transformador` y especializaciones para filtrar centrales, convertir de formato ancho a largo y construir variables derivadas. Estas clases se integran mediante Pipeline, que ejecuta una secuencia de transformaciones utilizando un contrato común.

A continuación se presentan las evidencias de herencia, polimorfismo y encapsulamiento en el proyecto, con su respectivo fragmento de código. En el caso de polimorfismo y encapsulamiento, ambas evidencias se destacan en la clase pipeline.

- **Herencia:** `FiltradorPeriodo`, `FiltradorCentrales`, `TransformadorAnchoLargo` y `ConstructorVariablesDerivadas` heredan de `Transformador`. La clase base establece el contrato común mediante los métodos ``ajustar()`` y ``transformar()``, mientras que cada subclase implementa únicamente el comportamiento que distingue su etapa del procesamiento.

```

1 class Transformador:
2     """Clase base: define el contrato común de todo paso del pipeline.
3
4     Separa ajustar() (aprende parámetros y valida) de transformar()
5     (aplica la transformación ya aprendida), siguiendo el método
6     plantilla: el orden y el control de estado quedan fijos aquí, y
7     cada clase derivada solo implementa aprender() y aplicar().
8     """
9
10    def __init__(self):
11        self._parametros: dict = {}
12        self._ajustado: bool = False
13
14    def ajustar(self, df: pd.DataFrame) -> "Transformador":
15        self._parametros = self.aprender(df)

```

```

16         self._ajustado = True
17         return self
18
19     def transformar(self, df: pd.DataFrame) -> pd.DataFrame:
20         if not self._ajustado:
21             raise RuntimeError("Hay que ajustar antes de transformar.")
22         return self.aplicar(df.copy()) # copia: no se muta la entrada
23
24     def aprender(self, df: pd.DataFrame) -> dict:
25         raise NotImplementedError("Cada clase derivada debe implementarlo.")
26
27     def aplicar(self, df: pd.DataFrame) -> pd.DataFrame:
28         raise NotImplementedError("Cada clase derivada debe implementarlo.")
29
30 class FiltradorPeriodo(Transformador): # herencia
31     """Delimita el dataset al periodo temporal definido para el análisis."""
32
33     def __init__(self, fecha_inicio: str, fecha_fin: str):
34         super().__init__()
35         self.fecha_inicio = fecha_inicio
36         self.fecha_fin = fecha_fin
37
38     def aprender(self, df: pd.DataFrame) -> dict:
39         if "Fecha" not in df.columns:
40             raise KeyError("La columna 'Fecha' no existe en el dataset.")
41
42         return {
43             "fecha_inicio": self.fecha_inicio,
44             "fecha_fin": self.fecha_fin,
45         }
46
47     def aplicar(self, df: pd.DataFrame) -> pd.DataFrame:
48         return filtrar_periodo(
49             df,
50             self.fecha_inicio,
51             self.fecha_fin,
52         )
53
54 class FiltradorCentrales(Transformador): # herencia
55     """Delimita el dataset a las centrales del alcance del proyecto."""
56
57     def __init__(self, centrales: list[str]):
58         super().__init__()
59         self.centrales = centrales
60
61     def aprender(self, df: pd.DataFrame) -> dict:
62         if "Central" not in df.columns:
63             raise KeyError("La columna 'Central' no existe en el dataset.")
64         presentes = sorted(set(df["Central"].unique()) & set(self.centrales))
65         return {"centrales_presentes": presentes}
66
67     def aplicar(self, df: pd.DataFrame) -> pd.DataFrame:
68         return filtrar_centrales(df, self.centrales)

```

- **Polimorfismo:** Pipeline recorre los distintos transformadores utilizando la misma interfaz `ajustar()/transformar()`, sin necesitar consultar el tipo concreto de cada objeto. Esto permite incorporar nuevas transformaciones compatibles sin modificar la lógica general del pipeline.
- **Encapsulamiento:** Cada clase concentra el estado y el comportamiento correspondiente a su responsabilidad. Los parámetros y el estado de ajuste se administran dentro de los objetos, evitando que otros componentes deban manipular directamente su lógica interna.

```
1 class Pipeline:
2     """Compone una lista de Transformador y los ejecuta en orden.
3
4     El bucle recorre self._pasos sin preguntar nunca qué clase
5     concreta es cada paso: solo invoca ajustar()/transformar(), que
6     cada Transformador implementa a su manera. Esa ausencia de
7     condicionales sobre el tipo es la evidencia de polimorfismo.
8     Agregar un cuarto paso no exige modificar esta clase (principio
9     de abierto y cerrado).
10    """
11
12    def __init__(self, pasos: list[Transformador]):
13        self._pasos = list(pasos)
14        self._ajustado = False
15
16    def ajustar(self, df: pd.DataFrame) -> "Pipeline":
17        intermedio = df.copy()
18        for paso in self._pasos:          # polimorfismo
19            intermedio = paso.ajustar(intermedio).transformar(intermedio)
20        self._ajustado = True
21        return self
22
23    def transformar(self, df: pd.DataFrame) -> pd.DataFrame:
24        if not self._ajustado:            # encapsulamiento
25            raise RuntimeError("Hay que ajustar antes de transformar.")
26        resultado = df.copy()
27        for paso in self._pasos:
28            resultado = paso.transformar(resultado)
29        return resultado
```

La ventaja de este diseño es que el flujo general no depende de una transformación específica. Cada clase puede modificar su implementación manteniendo el mismo contrato, lo que favorece la alta cohesión y bajo acoplamiento (vistos en el punto 6.1), y facilita extensión, reutilización y pruebas. La POO se utiliza así como mecanismo de organización del problema y no sólo como requisito formal.

7.2 Patrón Strategy para agregación temporal

El módulo `agregacion_temporal.py` implementa una familia de estrategias con el método común `agregar()`. `CaracterizadorTemporal` recibe una estrategia y delega en ella la agregación, permitiendo cambiar la escala temporal sin condicionales múltiples (`if/elif`) en una única función. Se implementan estrategias horaria, diaria, mensual y por día de semana.

Estrategia	Filas esperadas	Interpretación
Horaria	72	3 centrales × 24 horas
Diaria	729	3 centrales × 243 fechas
Mensual	24	3 centrales × 8 meses
Día de semana	21	3 centrales × 7 categorías

Tabla 12. Estrategias temporales implementadas y dimensiones resultantes.

El patrón Strategy permite cambiar la estrategia temporal sin modificar el objeto que la consume. Esta solución materializa polimorfismo y extensibilidad, y deja preparado el proyecto para incorporar nuevos niveles de agregación si el análisis posterior lo requiere.

VIII. Documentación de arquitectura

8.1 Arquitectura lógica

La arquitectura separa datos, notebooks, módulos y documentación. `data/raw` conserva la fuente; `data/processed` contiene el resultado transformado; `F1/`, `F2/` y `F3/` organizan notebooks por fase; `src/` contiene lógica reutilizable; y `docs/` mantiene documentación y evidencias.

```
Proyecto/  
├─ F1/F1_definicion.ipynb  
├─ F2/F2_preparacion_datos.ipynb  
├─ F3/F3_nucleo_algoritmico.ipynb  
├─ data/raw/generacion_real_cen_ene_ago_2026.csv  
├─ data/processed/dataset_cen_centrales_cmpc_ene_ago_2026.csv  
├─ docs/  
├─ src/{carga,transformacion,validacion,nucleo_poo,agregacion_temporal,secuencias,exportacion}.py  
└─ README.md · requirements.txt · .gitignore
```

IX. Repositorio GitHub

El repositorio mantiene trazabilidad mediante Git y GitHub, con ramas de trabajo, pull requests, commits

descriptivos y mailmap para consolidar identidades. La evolución de F3 incluye la incorporación del núcleo algorítmico, documentación, benchmarks, mejoras de retroalimentación y el refactor que añadió filtrado temporal y reutilización de funciones.

Referencia	Aporte
PR #5	Integración inicial del núcleo algorítmico y análisis de complejidad.
PR #6	Mejoras de reproducibilidad y benchmark completo.
2a7372f	Refactor POO e incorporación de FiltradorPeriodo.
99ff32b	Ajuste de .gitignore para copia local redundante.

Tabla 13. Aportes al repositorio en Fase 3.

Repositorio: [Proyecto-generacion-electrica-centrales-CMPC](#)

X. Notebooks ejecutables

F3/F3_nucleo_algoritmico.ipynb integra carga, pipeline POO, validaciones, comparación de transformaciones, algoritmos iterativo y recursivo, benchmarks y estrategias de agregación. La ejecución final fue verificada desde un kernel reiniciado y completó todas las celdas sin errores. F1 y F2 permanecen como antecedentes reproducibles de definición y preparación.

Notebook	Rol	Estado
F1_definicion.ipynb	Problema, objetivos, alcance y entorno reproducible.	Ejecutable
F2_preparacion_datos.ipynb	Carga, preparación, transformación y validación.	Ejecutable
F3_nucleo_algoritmico.ipynb	Algoritmos, eficiencia, POO, Strategy y pruebas.	Validado

Tabla 14. Notebooks ejecutables en repositorio.

XI. Bibliografía

La organización del informe sigue los apartados y criterios de la evaluación sumativa de Fase 3 (Universidad Andrés Bello, 2026b). Las fuentes combinan material docente, documentación técnica oficial y bibliografía académica.

Coordinador Eléctrico Nacional. (2022). Diccionario de datos Web SIP (B43-DIN-04). <https://www.coordinador.cl/wp-content/uploads/2022/11/B43-DIN-04-Diccionario-de-Datos-Web-SIP.pdf>

Coordinador Eléctrico Nacional. (2026). Generación real. <https://www.coordinador.cl/operacion/graficos/operacion-real/generacion-real/>

Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). Introduction to algorithms (4th ed.). MIT Press.

Python Software Foundation. (2026a). The Python language reference: Execution model. Python 3.14 documentation. <https://docs.python.org/3/reference/executionmodel.html>

Python Software Foundation. (2026b). tracemalloc - Trace memory allocations. Python 3.14 documentation. <https://docs.python.org/3/library/tracemalloc.html>

The pandas development team. (2026). pandas.DataFrame.melt. pandas documentation. <https://pandas.pydata.org/docs/reference/api/pandas.DataFrame.melt.html>

Universidad Andrés Bello. (2026a). Programación orientada a objetos: conceptos, principios y buenas prácticas [Material docente del curso MCDI500 Programación para la Ciencia de Datos].

Universidad Andrés Bello. (2026b). Avance del proyecto Fase 3 - Núcleo algorítmico, eficiencia e implementación orientada a objetos [Pauta de evaluación sumativa, MCDI500].

Anexo A. Síntesis de resultados técnicos

Indicador	Resultado
Dataset bruto	359.891 registros × 33 columnas
Dataset final	17.496 observaciones × 13 columnas
Cobertura	01-01-2026 a 31-08-2026
Centrales	TER CMPC Laja, Pacífico y Santa Fe
Ceros	4.438 (25,37%)
melt vs iterrows	6,1× más rápido en la ejecución registrada
Vectorizado vs iterativo	≈16,6× más rápido; +2,94% memoria trazada
Secuencias de ceros	Laja 211; Pacífico 76; Santa Fe 196
Reproducibilidad	Restart Kernel + Run All sin errores

Tabla 15. Síntesis de resultados técnicos Fase 3.

Nota metodológica: los tiempos y mediciones de memoria corresponden a ejecuciones concretas y pueden variar según hardware, sistema operativo, versión de Python y carga del equipo. Se utilizan como evidencia del comportamiento relativo de las implementaciones, no como valores universales.